Práctica 6: Estructuras de datos en R

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es aprender a utilizar las principales estructuras de datos disponibles en R. Al finalizarla, el estudiante será capaz de crear, consultar y modificar vectores, matrices y `data.frame`.

2. Secuencias

Una herramienta fundamental al trabajar con estructuras de datos es la generación de secuencias. La expresión

```
m:n
```

genera todos los enteros entre m y n. Esto es, $m, m+1, m+2, \ldots, n$. Por ejemplo:

```
> 3:10
```

genera:

```
[1]  3  4  5  6  7  8  9 10
```

También podemos utilizar `seq()`. Las siguientes instrucciones:

```
> seq(0, 1, by = 0.1)
> seq(2, 20, by = 2)
> seq(10, 1, by = -1)
```

tienen como resultado las siguientes secuencias:

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
[1]  2  4  6  8 10 12 14 16 18 20
[1] 10  9  8  7  6  5  4  3  2  1
```

Ejercicio 1. Utiliza secuencias para generar:

- los enteros del 1 al 20;
- los pares del 2 al 40;
- los múltiplos de 5 entre 5 y 100;
- los enteros del 20 al 1 en orden decreciente;
- los números 0, 0.25, 0.5, 0.75, ..., 5.

3. Vectores

Un vector es una colección ordenada de elementos del mismo tipo. Una forma habitual de crearlo es mediante la función `c()`:

```
notas <- c(7, 8.5, 6, 9, 5.5)
```

Si queremos ver su contenido por pantalla, entonces:

```
> notas
```

```
[1] 7.0 8.5 6.0 9.0 5.5
```

Un vector no está formado necesariamente de números. Podemos definir vectores como los siguientes:

```
saludo <- c("Buenos días", "Buenas tardes", "Buenas noches")
```

Si queremos ver su contenido por pantalla, entonces:

```
> saludo
```

```
[1] "Buenos días"   "Buenas tardes" "Buenas noches"
```

Que los elementos de un vector sean del mismo tipo tiene ciertas implicaciones. Por ejemplo, en el vector:

```
perfil <- c("Juan", 45, 83, 1.73)
```

que contiene información sobre Juan, como su edad (45 años), su peso (83 kg) o su altura (1.73 m), en realidad contiene cuatro cadenas de texto:

```
> perfil
```

```
[1] "Juan" "45"   "83"   "1.73"
```

Podemos usar secuencias para generar vectores:

```
> x <- c(seq(1,10,3))
> x
```

```
[1]  1  4  7 10
```

Ejercicio 2. Crea los siguientes vectores:

a) El vector que contiene todos los enteros del 1 al 100.

b) El vector que contiene todos los pares entre 2 y 200.

c) El vector que contiene los números 0.00, 0.01, 0.02, 0.03, 0.04, . . . , 1.

d) Los números de 1 al 100 en orden descendente.

3.1. Funciones sobre vectores

R implementa muchas funciones sobre vectores. Algunas de las más importantes son las siguientes:

Función	Descripción
`length`	Devuelve la longitud de un vector.
`min`	Devuelve el mínimo de un vector.
`max`	Devuelve el máximo de un vector.
`range`	Devuelve el rango (mínimo y máximo) de un vector.
`which.min`	Devuelve la posición del valor mínimo en el vector.
`which.max`	Devuelve la posición del valor máximo en el vector.
`sum`	Devuelve la suma de los elementos del vector.
`prod`	Devuelve el producto de los elementos del vector.
`mean`	Devuelve la media de los elementos del vector.
`rev`	Invierte el orden de los elementos de un vector.
`sort`	Ordena los elementos de un vector.

Ejercicio 3. Durante una semana, a las 12:00, se midió la temperatura en la calle. Los valores obtenidos fueron:

$$18,\ 21,\ 19,\ 23,\ 25,\ 22,\ 20.$$

Crea un vector llamado `temperaturas` que contenga dichas temperaturas y muestra:

a) el número de temperaturas;

b) la temperatura máxima y mínima;

c) la temperatura media;

d) las temperaturas ordenadas de menor a mayor;

e) las temperaturas ordenadas de mayor a menor.

3.2. Accediendo y modificando elementos de un vector

Podemos acceder a sus elementos mediante índices: para acceder al elemento de la posición i-ésima, utilizamos corchetes `[i]`. **En R, el índice del primer elemento del vector es el 1**, a diferencia de muchos otros lenguajes de programación donde el índice del primer elemento es el 0. Por ejemplo,

```
> notas <- c(7, 8.5, 6, 9, 5.5)
> notas[3]
```

```
[1] 6
```

También podemos seleccionar varios elementos utilizando un vector de índices:

```
> notas[c(1,3,5)]
```

```
[1] 7.0 6.0 5.5
```

Con índices negativos, accedemos a todos los elementos excepto a aquellos indicados:

```
> notas[c(-1,-4)]
```

```
[1] 8.5 6.0 5.5
```

Podemos modificar los elementos del vector:

```
> notas[3] <- 2.3
> notas
```

```
[1] 7.0 8.5 2.3 9.0 5.5
```

Ejercicio 4. Dado el vector

```
x <- c(4, 7, 2, 9, 12, 5, 8, 3)
```

obtén:

a) El segundo elemento.

b) Los elementos tercero, cuarto y quinto, todos al mismo tiempo.

c) Todos los elementos del vector menos el segundo y el último.

d) Los tres últimos elementos del vector.

3.3. Funciones y vectores

Los vectores pueden pasarse como argumento de funciones y ser devueltos por funciones. Podemos usar también bucles para recorrer los vectores. La función `seq_along()` genera una secuencia con tantos elementos como tenga un vector. Por ejemplo,

```
> x <- c(4, 7, 2, 9, 5, 3)
> seq_along(x)
```

genera la secuencia:

```
[1] 1 2 3 4 5 6
```

La siguiente función recorre un vector e imprime sus cuadrados:

```
imprime_cuadrados <- function(vector) {
  cuadrados <- "Los cuadrados de los elementos son:"
  for (i in seq_along(vector)) {
    cuadrados <- paste(cuadrados, vector[i]^2)
  }
  print(cuadrados)
}
```

Por ejemplo:

```
> x <- c(2,1,3,5,6)
> imprime_cuadrados(x)
```

```
[1] "Los cuadrados de los elementos son: 4 1 9 25 36"
```

Ejercicio 5. Define las funciones `minimo` y `maximo`, que devuelvan respectivamente el mínimo y el máximo. No puedes utilizar las funciones `min` y `max` ya implementadas en R. Compara los resultados con los obtenidos por dichas funciones. Pruébalo con el vector:

```
x <- c(4, 7, 2, 9, 12, 5, 8, 3)
```

Ejercicio 6. Escribe tres funciones llamadas `capicua_1`, `capicua_2` y `capicua_3` que, dado un vector, devuelvan `TRUE` si el vector es capicúa o `FALSE` en caso contrario. Deberán estar programadas de la siguiente forma:

1. Usando un bucle `for` y la función `rev` de R.
2. Usando un bucle `for`, pero sin usar la función `rev` de R.
3. Usando recursividad.

Comprueba que las tres dan la misma respuesta para los siguientes vectores:

```
x <- c(4, 4)
x <- c(4, 7)
x <- c(1, 7, 1, 5, 8, 3)
x <- c(1, 7, 5, 5, 7, 1)
```

3.4. Operaciones elemento a elemento sobre un vector

Muchas operaciones se realizan elemento a elemento. Por ejemplo, las instrucciones:

```
> notas <- c(7, 8.5, 6, 9, 5.5)
> notas + 1
> notas * 2
> notas^2
> notas >= 8
```

Tendrían como salidas:

```
[1]  8.0  9.5  7.0 10.0  6.5
[1] 14 17 12 18 11
[1] 49.00 72.25 36.00 81.00 30.25
[1] FALSE  TRUE FALSE  TRUE FALSE
```

Ejercicio 7. Crea una función que se llame `aprobados`, que reciba un vector de notas del 0 al 10, y devuelva un vector de valores booleanos, donde `TRUE` significa que la nota en esa posición es mayor o igual que 5, y `FALSE` en caso contrario. Pruébalo con los siguientes valores (haciendo una única llamada a la función):

$$8,\ 2,\ 9,\ 3,\ 5,\ 2,\ 0.$$

Ejercicio 8. Dado un vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ de $\mathbb{R}^n$, se define su **norma euclídea** como:

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Crea una función que se llame `norma_euclidea` que devuelva el valor de dicha norma. Calcula la norma del vector $\mathbf{x} = (1.0, 2.5, -1.2, -4.8, 5.7, 3.0)$.

4. Matrices

Una matriz es una estructura de datos bidimensional cuyos elementos son del mismo tipo. Está organizada en filas y columnas. En R podemos crear matrices mediante la función `matrix()`. Por ejemplo, la siguiente instrucción crea una matriz con dos filas y tres columnas:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)
```

Por defecto, R rellena las matrices por columnas. Por tanto, la matriz generada con la instrucción anterior es:

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Si queremos rellenar la matriz por filas, podemos utilizar el argumento `byrow = TRUE`:

```
m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3, byrow = TRUE)
```

En este caso, la matriz obtenida es:

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

4.1. Acceso a los elementos

Para acceder a un elemento de una matriz se utilizan dos índices entre corchetes: el primero indica la fila y el segundo, la columna. Por ejemplo, `m[2,3]` accede al elemento de la segunda fila y la tercera columna. Si se omite uno de los índices, R selecciona toda la fila o columna correspondiente. Por ejemplo, las instrucciones:

```
> m <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)
> m[2, 3]   # Elemento de la fila 2 y columna 3
> m[1, ]    # Primera fila
> m[, 2]    # Segunda columna
```

tiene como resultado, respectivamente:

```
[1] 6
[1] 1 2 3
[1] 2 5
```

Podemos modificar un elemento indicando su fila y columna, del mismo modo que hacemos con los vectores:

```
> m[1, 2] <- 20
> m
```

```
     [,1] [,2] [,3]
[1,]    1   20    3
[2,]    4    5    6
```

Ejercicio 9. Crea una matriz de 4 filas y 3 columnas que contenga los enteros del 1 al 12, rellenándola por filas. A continuación, realiza las siguientes operaciones:

1. muestra el elemento de la tercera fila y segunda columna;
2. muestra la primera fila;
3. muestra la tercera columna;
4. cambia el elemento de la fila 4, columna 1 por el número -8 y muestra la matriz resultante.

4.2. Modificar y operar con matrices

Las operaciones aritméticas habituales sobre una matriz se aplican elemento a elemento. Por ejemplo, `m * 2` multiplica cada elemento por dos, mientras que `m + 1` suma uno a cada elemento.

```
m * 2
m + 1
m^2
```

Es importante distinguir estas operaciones de la multiplicación matricial, que se realiza con el operador `%*%` y requiere dimensiones compatibles.

4.3. Funciones sobre matrices

R implementa muchas funciones sobre matrices. Algunas de las más importantes son las siguientes:

Función	Descripción
`nrow`	Devuelve el número de filas
`ncol`	Devuelve el número de columnas
`dim`	Devuelve las dimensiones de la matriz
`rowSums`	Calcula la suma de cada fila
`colSums`	Calcula la suma de cada columna
`rowMeans`	Calcula la media de cada fila
`colMeans`	Calcula la media de cada columna
`sum`	Suma todos los elementos de la matriz
`diag`	Devuelve un vector con los elementos de la diagonal
`t`	Devuelve la matriz traspuesta
`det`	Devuelve el determinante de la matriz

Ejercicio 10. Crea la siguiente matriz:

```
m <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
```

Calcula:

a) la suma de todos sus elementos;

b) la suma de cada fila;

c) la suma de cada columna;

d) la media de cada fila;

e) la media de cada columna;

f) su matriz traspuesta;

g) el vector que contiene su diagonal.

4.4. Recorrer una matriz

Podemos recorrer una matriz mediante dos bucles `for` anidados, uno para las filas y otro para las columnas. Las funciones `nrow()` y `ncol()` permiten adaptar los bucles a matrices de distintas dimensiones. El siguiente código imprime todos los elementos de una matriz. Fíjate en cómo recorre la matriz.

```r
for (i in 1:nrow(m)) {
    for (j in 1:ncol(m)) {
        print(m[i, j])
    }
}
```

Este ejemplo presupone que la matriz tiene al menos una fila y una columna. Para escribir código que también funcione con dimensiones vacías, se pueden usar secuencias como `seq_len(nrow(m))` y `seq_len(ncol(m))`.

Ejercicio 11. Escribe una función `suma_matriz(m)` que reciba una matriz y devuelva la suma de todos sus elementos. No puedes utilizar `sum()` ni `rowSums()` o `colSums()`; debes recorrer la matriz mediante dos bucles anidados. Pruébalo con la matriz:

$$M = \begin{pmatrix} 1 & 6 & 2 & 8 \\ 1 & -2 & -6 & 7 \\ -1 & -3 & 6 & 1 \\ -8 & -2 & 3 & 1 \end{pmatrix}$$

Compara el resultado con el que devuelve la función que implementa R para este propósito.

Ejercicio 12. Escribe una función `diagonal(m)` que devuelva un vector con los elementos de la diagonal principal de una matriz cuadrada. No puedes utilizar la función `diag()`.

$$M = \begin{pmatrix} -2 & 5 & -1 & 9 \\ 3 & -2 & 4 & -2 \\ 1 & -6 & 6 & 1 \\ -5 & 6 & -7 & -3 \end{pmatrix}$$

Compara el resultado con el que devuelve la función que implementa R para este propósito.

Ejercicio 13. Crea una función `es_simetrica(m)` que devuelva `TRUE` si la matriz cuadrada `m` es simétrica y `FALSE` en caso contrario. Una matriz es simétrica si el elemento de la fila i y columna j coincide con el de la fila j y columna i, para todas las parejas de índices. No utilices la función `t()` para resolver el ejercicio. Si la matriz no es cuadrada, debes mostrar un mensaje por pantalla que advierta de ello y finalizar la función sin devolver nada. A continuación, señala cuáles de las siguientes matrices son simétricas:

a) $M_1 = \begin{pmatrix} -2 & 3 & 9 \\ 3 & -2 & -2 \\ 1 & -6 & 1 \end{pmatrix}$.

b) $M_2 = \begin{pmatrix} -1 & 5 & -1 \\ 5 & -2 & -6 \\ -1 & -6 & 9 \end{pmatrix}$.

c) $M_3 = \begin{pmatrix} -1 & 5 & -3 & 9 \\ 3 & -2 & 4 & -8 \\ 1 & -6 & 3 & 1 \\ 4 & 5 & -3 & -8 \end{pmatrix}$.

d) $M_4 = \begin{pmatrix} 6 & 3 & 9 & -1 \\ 3 & -2 & -6 & 6 \\ 9 & -6 & 5 & -7 \\ -1 & 6 & -7 & 6 \end{pmatrix}$.

¿Cómo podrías comprobarlo con la función `t()`? ¿Obtienes los mismos resultados que con tu función?

5. Data frames

Un `data.frame` es una estructura tabular que permite almacenar datos organizados en filas y columnas. Cada columna puede contener un tipo de dato distinto, pero todas las columnas deben tener la misma longitud. Por ejemplo, podemos guardar el nombre, la edad y la nota de varios alumnos en un mismo `data.frame`.

```
alumnos <- data.frame(
  nombre = c("Ana", "Luis", "Marta", "Pablo"),
  edad = c(19, 21, 20, 22),
  nota = c(7.5, 5.0, 8.5, 6.0)
)
```

Para mostrar el contenido, basta con escribir el nombre del objeto:

```
> alumnos
```

```
  nombre edad nota
1    Ana   19  7.5
2   Luis   21  5.0
3  Marta   20  8.5
4  Pablo   22  6.0
```

Cada fila representa una observación, esto es, un alumno, y cada columna una variable.

5.1. Acceder a filas y columnas

Podemos acceder a una columna mediante el operador `$` o mediante índices. Para seleccionar una fila y una columna concretas, usamos dos índices entre corchetes: el primero corresponde a la fila y el segundo a la columna.

```
alumnos$nota        # Columna de notas
alumnos[2, ]        # Segunda fila
alumnos[, 3]        # Tercera columna
alumnos[2, 3]       # Fila 2, columna 3
alumnos[, "edad"]   # Columna llamada edad
```

Por ejemplo:

```
> alumnos$nombre
```

```
[1] "Ana"   "Luis"  "Marta" "Pablo"
```

Las funciones `nrow()` y `ncol()` devuelven el número de filas y columnas, respectivamente. La función `names()` devuelve los nombres de las columnas. En nuestro ejemplo:

```
> nrow(alumnos)
```

```
[1] 4
```

```
> ncol(alumnos)
```

```
[1] 3
```

```
> names(alumnos)
```

```
[1] "nombre" "edad"   "nota"
```

Ejercicio 14. Crea un `data.frame` llamado `libros` con información de los siguientes libros:

1. G. H. Hardy *A Course of Pure Mathematics* (1908), 446 páginas.
2. T. M. Apostol, *Mathematical Analysis* (1957), 492 páginas.
3. M. Spivak, *Calculus* (1967), 680 páginas.
4. S. Lang, *Introduction to Linear Algebra* (1970), 293 páginas.
5. J. R. Munkres, *Topology* (1975), 537 páginas.

Para cada libro, guarda el título, el autor, el año de publicación y el número de páginas. Muestra:

a) el número de libros;

b) el número de columnas (variables);

c) el título del tercer libro;

d) la información completa del segundo libro;

e) los títulos de todos los libros.

5.2. Añadir y modificar columnas

Podemos crear una nueva columna asignándole un vector de la misma longitud que el número de filas. También podemos modificar los valores de una columna existente.

```
> alumnos$nota[1] <- 8
> alumnos$aprobado <- alumnos$nota >= 5
> alumnos
```

```
  nombre edad nota aprobado
1    Ana   19  8.0     TRUE
2   Luis   21  5.0     TRUE
3  Marta   20  8.5     TRUE
4  Pablo   22  6.0     TRUE
```

En la primera instrucción se añade una columna lógica llamada `aprobado`. Sus valores son `TRUE` o `FALSE`, según se cumpla o no la condición.

Ejercicio 15. Sobre el `data.frame` `alumnos` del ejemplo, realiza las siguientes tareas:

1. Los alumnos han realizado un examen. Ana ha sacado un 9.5, Luis ha sacado un 2, Marta ha sacado un 3.5 y Pablo ha sacado un 4. Añade una columna `nota2` con una segunda nota para cada alumno;
2. una columna `media` con la media de las dos notas;
3. una columna lógica `aprobado` que indique si la media es mayor o igual que 5.

Muestra el `data.frame` completo y comprueba que los valores calculados son correctos.

5.3. Seleccionar filas mediante condiciones

Podemos seleccionar las filas que cumplen una condición utilizando un vector lógico. Por ejemplo:

```
> notables_y_sobresalientes <- alumnos[alumnos$media >= 7, ]
> notables_y_sobresalientes
```

```
  nombre edad nota aprobado nota2 media
1    Ana   19    8     TRUE   9.5  8.75
```

```
> mayores <- alumnos[alumnos$edad > 20, ]
> mayores
```

```
  nombre edad nota aprobado nota2 media
2   Luis   21  5.0    FALSE   2.0  3.50
4  Pablo   22  6.0     TRUE   4.0  5.00
```

La expresión situada antes de la coma determina qué filas se seleccionan. Al dejar vacía la parte posterior a la coma, se conservan todas las columnas.

Ejercicio 16. Utilizando el `data.frame` `libros` del ejercicio 14, crea y muestra:

a) un `data.frame` con los libros posteriores a 1960;

b) un `data.frame` con los libros de más de 500 páginas;

c) un `data.frame` con los libros posteriores a 1960 que además tengan más de 500 páginas.

5.4. Funciones sobre data frames

Podemos utilizar las funciones vistas en el apartado de vectores sobre las columnas de un `data.frame`. Por ejemplo:

```
> sort(alumnos$nota)
```

```
[1] 5.0 6.0 8.0 8.5
```

```
> mean(alumnos$nota)
```

```
[1] 6.875
```

```
> max(alumnos$nota)
```

```
[1] 8.5
```

Podemos utilizar la función `which.max()`, que devuelve la posición del primer valor máximo, para obtener la información de la fila correspondiente.

```
> posicion <- which.max(alumnos$nota)
> alumnos[posicion, ]
```

```
  nombre edad nota aprobado nota2 media
3  Marta   20  8.5     TRUE   3.5     6
```

Ejercicio 17. Pensando en el `data.frame libros` del ejercicio 14, escribe las siguientes funciones:

1. `mas_reciente(datos)`, que reciba un `data.frame` con una columna llamada `año` y devuelva la fila completa correspondiente al elemento más reciente de ese conjunto de datos;
2. `mas_antiguo(datos)`, que reciba un `data.frame` con una columna llamada `año` y devuelva la fila completa correspondiente al elemento más antiguo de ese conjunto de datos;
3. `mas_paginas(datos)`, que reciba un `data.frame` con una columna llamada `paginas` y devuelva la fila completa correspondiente al elemento con mayor número de páginas en ese conjunto de datos;
4. `menos_paginas(datos)`, que reciba un `data.frame` con una columna llamada `paginas` y devuelva la fila completa correspondiente al elemento con menor número de páginas de ese conjunto de datos;
5. `total_paginas(datos)`, que reciba un `data.frame` con una columna llamada `paginas` y devuelva el total de páginas que suman todos los elementos de ese conjunto de datos;
6. `media_paginas(datos)`, que reciba un `data.frame` con una columna llamada `paginas` y devuelva la media de páginas de todos los elementos de ese conjunto de datos,

Prueba las funciones con el `data.frame libros`.

6. Un último ejercicio

Recuerda documentar tu práctica.

Ejercicio 18. Escribe un fichero `README.md` para tu directorio donde guardarás la práctica 6. El aspecto debe ser similar al de los ficheros `README.md` que hiciste para las prácticas 1, 3 y 5.

7. Entrega y consejos

Sube los ficheros con los ejercicios a tu GitHub en la fecha acordada con el profesor. Para esta práctica:

1. guarda tus scripts en la carpeta correspondiente;
2. utiliza nombres de fichero claros y consistentes;
3. comenta el código para facilitar su comprensión;
4. comprueba que puedes ejecutar de nuevo cada script desde el principio;
5. evita depender de instrucciones que hayan quedado solamente en la consola.

Idea clave: los vectores permiten trabajar con colecciones de valores, las matrices organizan datos en filas y columnas y los `data.frame` permiten representar datos tabulares formados por distintas variables. Las secuencias permiten generar de forma sencilla conjuntos de valores que podemos recorrer, seleccionar y transformar.